



Tema:

Modelado de la Malla Curricular como Grafo Dirigido para el
Análisis de Prerrequisitos mediante BFS y DFS

Asignatura:

Estructura de Datos y Algoritmos II

Integrantes:

Andrés Merino, Mathew Verdezoto

Carrera:

Ciencia de Datos e Inteligencia Artificial

Ingeniero/a Encargado/a:

Lorena Recalde

Fecha Entrega:

31/05/2026

1. Introducción

El presente informe tiene como objetivo generar un sistema de dependencias académicas mediante la creación de un Grafo Dirigido Acíclico con la ayuda de Programación Orientada a Objetos en el lenguaje de Python. Mediante el modelo representamos las asignaturas de una malla curricular universitaria y cada uno con sus respectivos prerrequisitos que nos ayudaran para aplicar los algoritmos de búsqueda BFS y DFS. Mediante esto podremos analizar de manera organizada cuales son las rutas de más viables para los estudiantes, con esto también demostramos que las estructuras de datos son una herramienta muy útil para resolver problemas en el mundo real.

2. Objetivos

Objetivo General:

- Elaborar una aplicación en Python basada en POO que transforme una malla curricular a un grafo dirigido para el control de dependencias académicas, aplicando los algoritmos BFS y DFS para la exploración de rutas de estudio y cuál es la mejor ruta.

Objetivos Específicos:

- Elaborar un Grafo Dirigido Acíclico que represente a 20 nodos de las materias con sus prerrequisitos, generando la carga de los datos con un archivo JSON.
- Diseñar el sistema principal y los algoritmos BFS y DFS mediante POO y el uso de funciones en Python.
- Evaluar cuales fueron las rutas generadas para concluir cual tiene mayor eficiencia, todo esto acompañado de la visualización gráfica del recorrido final.

3. Definición del Problema Real

Todos los días los estudiantes de las diferentes universidades enfrentan el mismo problema al momento de organizar su ruta de matriculación, estos problemas ocurren por las restricciones por prerrequisitos en la malla curricular. Si llega a existir errores al escoger materias en los primeros semestres puede causar problemas como el bloqueo al acceso de asignaturas avanzadas y esto puede causar que el estudiante se retrase en su proceso de titulación. El problema radica en la falta de materiales visuales y validación de estas dependencias a largo plazo usando documentos ya establecidos. Cuando se modela la malla como un grafo dirigido, estamos cambiando este problema administrativo a un problema computacional, de esta manera podemos automatizar el proceso de la validación de prerrequisitos y así los estudiantes pueden encontrar rutas viables de estudio mediante algoritmos de búsqueda. Aplicando cada uno con un paradigma diferente, siendo BFS para encontrar la ruta más corta a una materia dada y DFS para visualizar todas las materias dependientes desde una materia dada, así sabiendo qué pasaría en caso de perder una materia y cómo esto afectaría a su recorrido académico,

4. Modelado del Problema como Grafo

Para modelar el problema de la planificación académica y validación de prerrequisitos nos basamos en la estructura y lógica de un Grafo Dirigido Acíclico. Esta elección de grafo la hicimos porque consideramos que es la mejor para representar estructuras organizadas.

Definimos un grafo de la siguiente manera:

$$G = (V, E)$$

Donde:

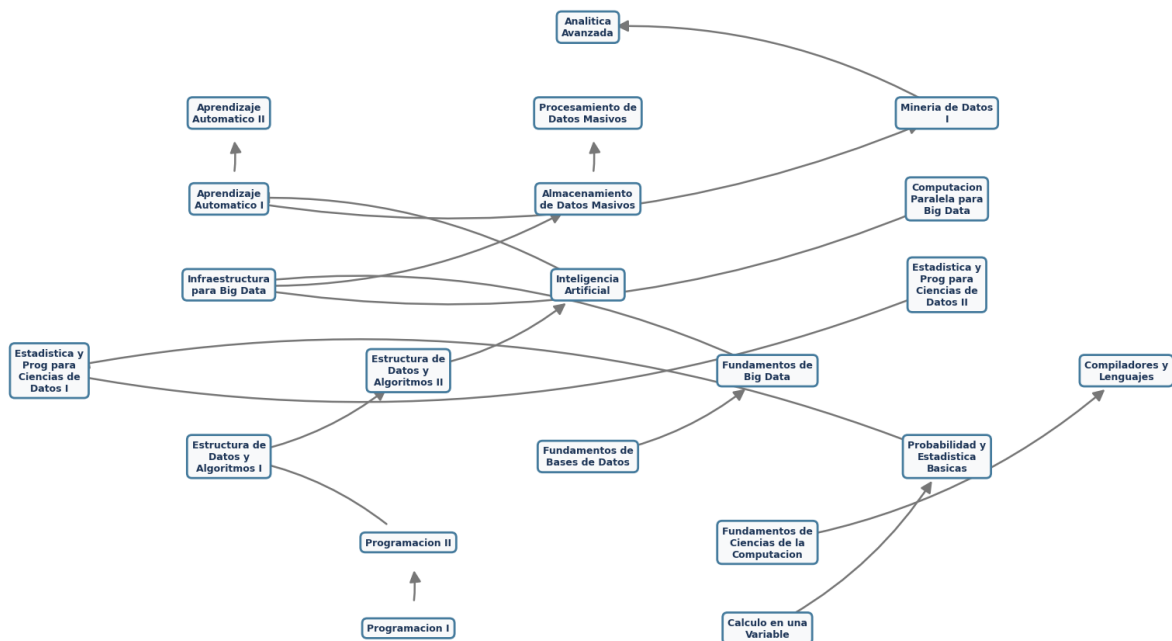
- **V:** Son el conjunto de vértices que representan a un conjunto específico de la malla curricular, en este caso nos basamos en la malla de la carrera de Ciencia de Datos e Inteligencia Artificial. Nuestro diseño cuenta 20 nodos en total. Todos los nodos contienen a una asignatura y cuenta con dos características principales:
 - **nombre:** Distintivo principal de la materia.
 - **nivel:** Es el semestre en el que está dicha asignatura dentro del plan curricular.
- **E:** Es el conjunto de aristas que se encargan de representar las relaciones que existen entre las asignaturas. Estas conexiones son dirigidas y se representan mediante flechas que se trazan desde un vértice materia origen hasta otro vector destino.

Figura 1. Malla Curricular Base

Nivel	Materia	Prerrequisitos
1	Álgebra Lineal	NA
1	Cálculo en una Variable	NA
1	Mecánica Newtoniana	NA
1	Programación I	NA
1	Comunicación Oral y Escrita	NA
2	Ecuaciones Diferenciales Ordinarias	Álgebra Lineal, Cálculo en una Variable
2	Introducción a los Sistemas de Información	NA
2	Fundamentos de Ciencias de la Computación	NA
2	Programación II	Programación I
2	Arquitectura de Computadores	NA
2	Análisis Socioeconómico y Político del Ecuador	NA
3	Probabilidad y Estadística Básicas	Cálculo en una Variable
3	Sistemas Operativos	Programación I
3	Fundamentos de Bases de Datos	NA
3	Estructura de Datos y Algoritmos I	Programación II
3	Fundamentos de Redes y Conectividad	NA
4	Fundamentos de Big Data	Fundamentos de Bases de Datos
4	Estadística y Programación para Ciencias de Datos I	Probabilidad y Estadística Básicas
4	Compiladores y Lenguajes	Fundamentos de Ciencias de la Computación
4	Estructura de Datos y Algoritmos II	Estructura de Datos y Algoritmos I
4	Bases de Datos Distribuidas	Fundamentos de Bases de Datos
4	Asignatura de Artes y Humanidades	NA
4	Prácticas de Servicio Comunitario	NA
5	Infraestructura para Big Data	Fundamentos de Big Data
5	Estadística y Programación para Ciencias de Datos II	Estadística y Programación para Ciencias de Datos I
5	Computación Numérica	NA
5	Inteligencia Artificial	Estructura de Datos y Algoritmos II
5	Desarrollo y Mantenimiento de Software	NA
5	Asignatura de Economía y Sociedad	NA
5	Gestión Organizacional	NA
6	Almacenamiento de Datos Masivos	Infraestructura para Big Data

6	Computación Paralela para Big Data	Infraestructura para Big Data
6	Aprendizaje Automático I	Inteligencia Artificial
6	Aplicaciones Web	Programación II
6	Tecnologías de Seguridad	Fundamentos de Redes y Conectividad
7	Procesamiento de Datos Masivos	Almacenamiento de Datos Masivos
7	Aprendizaje Automático II	Aprendizaje Automático I
7	Minería de Datos I	Aprendizaje Automático I
7	Seguridad y Privacidad de Datos	Tecnologías de Seguridad
7	User Experience	NA
7	Gestión de Procesos y Calidad	NA
7	Ingeniería Financiera	NA
8	Visualización de Datos	Procesamiento de Datos Masivos
8	Servicios en la Nube para Big Data	NA
8	Minería de Datos II	Minería de Datos I
8	Analítica Avanzada	Minería de Datos I
8	Profesionalismo en Informática	NA
8	Diseño de Trabajo de Integración Curricular / Prep. Examen	NA
9	Trabajo de Integración Curricular / Examen Complexivo	Diseño de Trabajo de Integración Curricular / Prep. Examen
9	Prácticas Laborales	NA
9	Aplicaciones Emergentes de Ciencia de Datos	Visualización de Datos
9	Administración de Proyectos de Ciencia de Datos	NA

Figura 2: Visualización gráfica del grafo mallas



5. Diseño de Clases

La arquitectura principal de nuestro sistema se lo elaboró en base a las reglas de la Programación Orientada a Objetos. Las propiedades críticas de las entidades las definimos como privados y las podemos ver mediante los decoradores `@property`.

La estructura del código se compone de tres clases principales que se relacionan de la siguiente forma:

Clase Arista

Se encarga de diseñar la relación unidireccional que existe entre dos materias de la malla curricular.

- **Atributos Privados:**
 - **__origen:** Es el identificador de la materia que se define como prerequisite.
 - **__destino:** Es el identificador de la materia dependiente.
- **Propiedades:**
 - **origen:** Se encarga de la lectura del nodo donde inicia la arista.
 - **destino:** Se encarga de la lectura del nodo final de la arista.

Clase Nodo

Separa los elementos individuales de la malla y crea sus propias conexiones de adyacencia.

- **Atributos Privados:**
 - **__id:** Se trata nombre de cada asignatura.
 - **__nivel:** Es el nivel académico que se encuentra en el plan de estudios.
 - **__aristas_salientes:** Se basa en una lista que se encarga de almacenar instancias de la clase Arista.
- **Propiedades:**
 - **id:** Muestra el nombre de la materia para que se puedan realizar consultas externas.
 - **nivel:** Expone el nivel en el que esta la asignatura.
- **Métodos Públicos:**
 - **agregar_adyacencia:** Se encarga de instanciar una nueva arista con el id del nodo en el que se encuentra como origen y el nodo que se marcó como destino, después la guarda en la lista de aristas salientes.
 - **obtener_vecinos:** Devuelve la lista estructurada con sus respectivos identificadores de los nodos alcanzables desde dicho vértice.

Clase Grafo

Su función principal es actuar como contenedor de la estructura y a su vez se encarga de controlar todo el sistema. Se encarga de organizar la estructura y ejecutar las búsquedas.

- **Atributos Públicos:**
 - **nodos:** Un diccionario que nos sirve de acceso público estructurado para facilitar la integración de un menú interactivo, también nos ayuda con lectura

del archivo JSON que contiene los datos de la malla y también genera la estructura de coordenadas con la librería NetworkX.

- **Métodos Públicos:**

- **agregar_nodo:** Este método se encarga de verificar que exista el vértice, de ser el caso que sea nuevo, instancia un objeto de la clase Nodo dentro del diccionario que recolecta estos datos.
- **agregar_arista:** Verifica y válida la presencia de nodos y aristas en el sistema y transfiere al nodo de inicio la tarea de registrar la adyacencia hasta el nodo destino.
- **bfs_camino_minimo:** Su lógica se basa en implementar el recorrido BFS mediante el uso de colas estructuradas para poder encontrar la mejor ruta, nos devuelve la secuencia y el orden en el que se exploraron los vértices.
- **dfs_camino:** Se basa en la lógica de DFS implementando la estructura de pilas, explora todos los caminos antes de retroceder y retorna la cadena de la ruta y el registro en el que hizo la búsqueda.

6. Implementación de BFS y DFS

Tabla 1. Implementación Lógica BFS y DFS

Característica	BFS	DFS
Estrategia de Exploración	Nivel por nivel. Examina todos los prerequisites entre nodos antes de profundizar.	Usamos backtracking. Se va hasta el final de una rama de prerequisites antes de empezar a retroceder.
Estructura de Datos Base	Cola y Queue. La implementamos mediante deque.	Pila y Stack. Se la implemento mediante una lista estándar de Python usando .append y .pop.
Lógica	Se garantiza encontrar el camino más corto.	Primero encuentra la ruta más viable, y que no necesariamente es la más ruta más corta.
Control de Inserción	Se realizó una inserción secuencial de los nodos vecinos que se encontraron en la cola.	Uso de la función reversed() al momento de hacer la inserción de vecinos para seguir la regla del orden lógico.
Datos Retornados	Se retorna la ruta exacta, la cantidad de movimientos y el orden final de la exploración.	Imprime la primera ruta válida en la ejecución y el orden secuencial de la malla.

7. Pruebas de ejecución de los scripts

Dentro de la aplicación, el primer módulo visualizado por el usuario es el menú interactivo para recorrer el grafo creado, dicho menú tendrá 5 opciones: explorar la lista de materias de la malla curricular (vuelta un grafo), ejecutar los algoritmos BFS o DFS, visualizar el grafo creado para la malla y salir de la aplicación, dentro de la figura 3 se observa la implementación

Figura 3. Menú interactivo al iniciar la aplicación

```
PS C:\Users\usuario\Desktop\EPN\Semestres\CuartoSemestre\EDA_II\Proyecto_Bim_1> & C:/Users/usuario/AppData/Local/Programs/Python/Python313/python.exe c:/Users/usuario/Desktop/EPN/Semestres/CuartoSemestre/EDA_II/Proyecto_Bim_1/main.py

EXPLORADOR INTELIGENTE DE GRAFOS
1. Ver lista de materias de la malla
2. Ejecutar algoritmo BFS
3. Ejecutar algoritmo DFS
4. Visualizar grafo de la malla
5. Salir
Elige una opción: █
```

Opciones:

1. Veremos la lista de materias disponibles en la malla curricular, las cuales son detalladas en la figura 1

Figura 3: Prueba de ejecución opción 1

```
Elige una opción: 1

--- MATERIAS DISPONIBLES ---
- Programacion I
- Calculo en una Variable
- Fundamentos de Ciencias de la Computacion
- Fundamentos de Bases de Datos
- Programacion II
- Probabilidad y Estadistica Basicas
- Estructura de Datos y Algoritmos I
- Estructura de Datos y Algoritmos II
- Fundamentos de Big Data
- Estadistica y Prog para Ciencias de Datos I
```

2. Realiza el recorrido BFS, primero pide una materia de entrada y otra de salida, en caso de que no encuentre camino el script enviará un mensaje de que el recorrido no se pudo realizar porque no encontró ruta, caso contrario realizará el recorrido BFS sin problema

Figura 4: Camino sin salida BFS

```
3. Ejecutar algoritmo DFS
4. Visualizar grafo de la malla
5. Salir
Elige una opción: 2

--- CONFIGURACIÓN BFS ---

(Escribir el nombre exacto de la materia)
Escribe la materia inicial (Start): Compiladores y Lenguajes
Escribe la materia final (Finish): Minería de Datos I

--- INICIANDO BFS DESDE 'Compiladores y Lenguajes' HASTA 'Minería de Datos I' ---

Estructura Inicial - Cola: [Compiladores y Lenguajes], Visitados: []

Nodo actual: Compiladores y Lenguajes
Estado de la cola: []
Nodos Marcados: ['Compiladores y Lenguajes']

Nodos Marcados: ['Compiladores y Lenguajes']

No existe camino desde 'Compiladores y Lenguajes' hasta 'Minería de Datos I'
```

Figura 5: ejecución normal BFS

```
--- INICIANDO BFS DESDE 'Programacion I' HASTA 'Inteligencia Artificial' ---

Estructura Inicial - Cola: [Programacion I], Visitados: []

Nodo actual: Programacion I
Estado de la cola: ['Programacion II']
Nodos Marcados: ['Programacion I']

Nodo actual: Programacion II
Estado de la cola: ['Estructura de Datos y Algoritmos I']
Nodos Marcados: ['Programacion I', 'Programacion II']

Nodo actual: Estructura de Datos y Algoritmos I
Estado de la cola: ['Estructura de Datos y Algoritmos II']
Nodos Marcados: ['Programacion I', 'Programacion II', 'Estructura de Datos y Algoritmos I']

Nodo actual: Estructura de Datos y Algoritmos II
Estado de la cola: ['Inteligencia Artificial']
Nodos Marcados: ['Programacion I', 'Programacion II', 'Estructura de Datos y Algoritmos I', 'Estructura de Datos y Algoritmos II']

Nodo actual: Inteligencia Artificial
Nodos Marcados: ['Programacion I', 'Programacion II', 'Estructura de Datos y Algoritmos I', 'Estructura de Datos y Algoritmos II', 'Inteligencia Artificial']

Se encontro destino

Ruta más corta: Programacion I -> Programacion II -> Estructura de Datos y Algoritmos I -> Estructura de Datos y Algoritmos II -> Inteligencia Artificial
Distancia: 4 arista(s)
Orden de exploración: Programacion I, Programacion II, Estructura de Datos y Algoritmos I, Estructura de Datos y Algoritmos II, Inteligencia Artificial
```

3. Realiza el recorrido DFS, al igual que en BFS notificará si no existe un camino, caso contrario ejecutará el algoritmo paso a paso normalmente

Figura 6: Camino sin salida DFS

```

Elige una opción: 3

--- CONFIGURACIÓN DFS ---

(Escribir el nombre exacto de la materia)
Escribe la materia inicial (Start): Fundamentos de Ciencias de la Computacion
Escribe la materia final (Finish): Estadística y Prog para Ciencias de Datos II

--- INICIANDO DFS DESDE 'Fundamentos de Ciencias de la Computacion' HASTA 'Estadística y Prog para Ciencias de Datos II' ---

Padres iniciales: {'Fundamentos de Ciencias de la Computacion': None}
READY (Pila): ['Fundamentos de Ciencias de la Computacion']

Nodo para usarse: Fundamentos de Ciencias de la Computacion
  Compiladores y Lenguajes: registrando padre
READY (Pila): ['Compiladores y Lenguajes']
Nodos Marcados: ['Fundamentos de Ciencias de la Computacion']

Nodo para usarse: Compiladores y Lenguajes
READY (Pila): []
Nodos Marcados: ['Fundamentos de Ciencias de la Computacion', 'Compiladores y Lenguajes']

Nodos Marcados: ['Fundamentos de Ciencias de la Computacion', 'Compiladores y Lenguajes']

No existe camino desde 'Fundamentos de Ciencias de la Computacion' hasta 'Estadística y Prog para Ciencias de Datos II'

```

Figura 7: ejecución normal DFS

```

--- INICIANDO DFS DESDE 'Programacion I' HASTA 'Estructura de Datos y Algoritmos II' ---

Padres iniciales: {'Programacion I': None}
READY (Pila): ['Programacion I']

Nodo para usarse: Programacion I
  Programacion II: registrando padre
READY (Pila): ['Programacion II']
Nodos Marcados: ['Programacion I']

Nodo para usarse: Programacion II
  Estructura de Datos y Algoritmos I: registrando padre
READY (Pila): ['Estructura de Datos y Algoritmos I']
Nodos Marcados: ['Programacion I', 'Programacion II']

Nodo para usarse: Estructura de Datos y Algoritmos I
  Estructura de Datos y Algoritmos II: registrando padre
READY (Pila): ['Estructura de Datos y Algoritmos II']
Nodos Marcados: ['Programacion I', 'Programacion II', 'Estructura de Datos y Algoritmos I']

Nodo para usarse: Estructura de Datos y Algoritmos II
Nodos Marcados: ['Programacion I', 'Programacion II', 'Estructura de Datos y Algoritmos I', 'Estructura de Datos y Algoritmos II']

Se encontro destino

Ruta encontrada: Programacion I -> Programacion II -> Estructura de Datos y Algoritmos I -> Estructura de Datos y Algoritmos II
Orden de exploración: Programacion I, Programacion II, Estructura de Datos y Algoritmos I, Estructura de Datos y Algoritmos II
Árbol DFS (Padres): {'Programacion I': None, 'Programacion II': 'Programacion I', 'Estructura de Datos y Algoritmos I': 'Programacion II', 'Estructura de Datos y Algoritmos II': 'Estructura de Datos y Algoritmos I'}

```

4. Abre una pestaña que nos muestra el grafo con el que estamos trabajando, visto en la figura 2, para generar este gráfico, se utilizó Para generar este gráfico, se utilizó la librería networkx para crear y manipular la estructura matemática de la red, almacenando las aristas y calculando las coordenadas lógicas de distribución (Hagberg, Schult & Swart, 2008).

El renderizado final en pantalla se ejecutó mediante matplotlib, un entorno 2D que recibió dichas coordenadas para dibujar los elementos visuales como formas, líneas y flechas (Hunter, 2007). Finalmente, para garantizar una correcta presentación de las etiquetas, se aplicó el módulo textwrap, el cual dividió los nombres largos de las asignaturas en múltiples líneas cortas para que se ajustaran visualmente al área de cada nodo sin desbordarse (Python Software Foundation, 2026).

5. Finaliza el programa y deja al usuario salir del menú

7. Comparativa DFS vs BFS

Como es bien conocido, los algoritmos DFS y BFS tienen paradigmas completamente distintos pese a cubrir la misma funcionalidad de recorrer un grafo ambos difieren desde las estructuras que usan, hasta el orden que visitan cada nodo, por lo tanto, ambos algoritmos tendrán una aplicación diferente al problema del recorrido de mallas curriculares, dichas diferencias son descritas en la tabla 2

Tabla 2: Comparación de BFS y DFS

Criterio	BFS	DFS
Estructura auxiliar utilizada	Cola (FIFO - Implementada con deque).	Pila (LIFO - Implementada con listas y <code>pop()</code> y diccionario de predecesores.
Estrategia	Explora por niveles (expansión a lo ancho).	Explora en profundidad (rama por rama hasta el final usando backtracking).
Funcionalidad con respecto al problema	Sirve para calcular la cantidad mínima de saltos o semestres necesarios para alcanzar una asignatura de niveles superiores. Cubre la necesidad de trazar la trayectoria académica más rápida y directa hacia una meta específica, optimizando el tiempo del estudiante.	Cubre la necesidad de prever el efecto negativo de reprobar una asignatura base, revelando instantáneamente todas las materias futuras que quedarán inhabilitadas por falta de ese prerrequisito.

8. Conclusiones

- El modelado de un problema del mundo real, como lo es una malla curricular universitaria, a través de un Grafo Dirigido Acíclico (DAG) demostró ser una factible para automatizar la validación de prerrequisitos, previniendo errores en la planificación académica de los estudiantes.
- La implementación del Tipo de Dato Abstracto (ADT) bajo el paradigma de Programación Orientada a Objetos (clases Grafo, Nodo y Arista) permitió un control estricto mediante encapsulamiento. Esto facilitó mantener la lógica algorítmica separada y limpia, permitiendo su fácil integración con librerías externas de visualización como NetworkX.

- El contraste de los algoritmos confirmó sus fundamentos teóricos: BFS probó ser la herramienta indispensable para trazar rutas óptimas (caminos más cortos) empleando colas, mientras que DFS resultó valioso para recorrer ramas completas de prerequisites de manera exhaustiva utilizando pilas.
- El sistema demostró robustez al implementar validaciones de frontera, logrando detectar de manera determinista los grafos desconectados (cuando se intenta ir a una materia de otra rama que no se conecta con otra), manteniendo un sistema consistente ante el diseño real de la malla curricular de la carrera

9. Referencias

- Aric A. Hagberg, Daniel A. Schult and Pieter J. Swart, “[Exploring network structure, dynamics, and function using NetworkX](#)”, in [Proceedings of the 7th Python in Science Conference \(SciPy2008\)](#), Gäel Varoquaux, Travis Vaught, and Jarrod Millman (Eds), (Pasadena, CA USA), pp. 11–15, Aug 2008
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95. Recuperado de <https://matplotlib.org/stable/>
- Python Software Foundation. (2026). textwrap — Text wrapping and filling. Python 3.14 Documentation. Recuperado de <https://docs.python.org/3/library/textwrap.html>
- Recalde, L. (2026a). Estructuras de Datos y Algoritmos II: Semana 5 [Archivo PDF]. Escuela Politécnica Nacional.
- Recalde, L. (2026b). Estructuras de Datos y Algoritmos II: Semana 3 [Archivo PDF]. Escuela Politécnica Nacional.

10. Declaración uso de IA

Sí he usado IA para la elaboración de este informe; sin embargo, declaro haber revisado y corregido los contenidos, de tal forma que soy responsable y entiendo lo descrito. Tomado de <https://investigauned.uned.es/como-declarar-el-uso-de-ia-en-trabajos-academicos/> (ver ejemplos en el sitio Web).

10.1 Herramienta utilizada: Gemini (versión Pro 3.1).

10.2 Propósito del uso: Búsqueda de fuentes científicas para la introducción y corrección ortográfica y gramatical del texto.

10.3 Prompts o instrucciones proporcionadas: "Mejora la redacción de este punto", "Revisa la ortografía y gramática de este texto y sugiere correcciones", "Tengo la siguiente idea para mi código,, enséñame cómo llegar a implementarla sin darme la solución para copiar y pegar", "tengo que replicar el siguiente código, enséñame a realizarlo sin copiar y pegar", "Ajusta el siguiente script de generación de grafo para que tenga estas características

visuales...”, “Valida que mi informe sea completo según la rúbrica”, “Del siguiente documento que detalla las relaciones de prerequisites de materias, conviértelos a un archivo json”.

10.4 Uso del contenido generado: Las referencias sugeridas sirvieron como guía para localizar los artículos originales. Se aplicaron sugerencias de puntuación y ortografía para mejorar la claridad.

10.5 Revisión y edición: Todas las fuentes bibliográficas fueron contrastadas empíricamente y las sugerencias de redacción se revisaron manualmente antes de su aplicación.

10.6 Limitaciones y consideraciones éticas: La herramienta no generó resultados, análisis ni conclusiones. Tampoco alteró el significado del texto original. Se asume la total responsabilidad y autoría del trabajo presentado.

Herramienta usada	Actividad en la que se usó (Prompts principales)	Qué modificó el equipo	Responsable
Gemini 3.1 Pro	Revisión y mejora de redacción: <i>"Mejora la redacción de este punto", "Revisa la ortografía y gramática..."</i>	Se evaluaron las sugerencias ortográficas y gramaticales, adaptando manualmente el tono formal del informe técnico.	Mathew Verdezoto
Gemini 3.1 Pro	Asistencia pedagógica en código: <i>"Tengo la siguiente idea para mi código... enséñame cómo llegar a implementarla", "tengo que replicar el siguiente código..."</i>	Se analizó la lógica teórica proporcionada por la IA y se estructuró el código manualmente desde cero, adaptándolo a las clases del proyecto.	Felipe Merino
Gemini 3.1 Pro	Ajustes de visualización: <i>"Ajusta el siguiente script de generación de grafo para que tenga estas características visuales..."</i>	Se integraron los parámetros de escalado y diseño sugeridos para NetworkX dentro de la función de visualización del sistema.	Felipe Merino
Gemini 3.1 Pro	Conversión de estructura de datos: <i>"Del siguiente</i>	Se validó la jerarquía del archivo	Mathew Verdezoto

	<i>documento... conviértelos a un archivo json"</i>	generado, asegurando que el JSON cargara correctamente las claves "materia", "nivel" y "prerrequisitos".	
Gemini 3.1 Pro	Validación de documentación: <i>"Valida que mi informe sea completo según la rúbrica"</i>	Se agregaron las secciones y tablas que la herramienta señaló como faltantes para asegurar el cumplimiento estricto de la rúbrica.	Mathew Verdezoto
Gemini 3.1 Pro	Validación de documentación: <i>"Valida que mi informe sea completo según la rúbrica"</i>	Se agregaron las secciones y tablas que la herramienta señaló como faltantes para asegurar el cumplimiento estricto de la rúbrica.	Mathew Verdezoto